

Session 8 : Rendering Pipeline (Partie 1)

💡 Objectifs de la session

Comprendre l'architecture du pipeline de rendu : du modèle 3D au pixel à l'écran.

Vue d'Ensemble du Pipeline

Étapes du pipeline graphique

1. **Application Stage (CPU)** : Le code (JS) prépare les données : matrices, paramètres, culling.
2. **Geometry Processing (GPU)** : Vertex Shader, Tessellation, Geometry Shader.
3. **Rasterization (GPU)** : Conversion des triangles en fragments (pixels).
4. **Pixel Processing (GPU)** : Fragment Shader, tests de profondeur, blending.
5. **Output Merger** : Écriture dans le framebuffer.

Vertex Shader en Détail

Responsabilités

Le Vertex Shader transforme chaque sommet :

- Object Space → World Space (matrice **world**)
- World Space → View Space (matrice **view**)
- View Space → Clip Space (matrice **projection**)
- $$MVP = P \times V \times W$$

Il peut aussi passer des données au Fragment Shader (normales, UVs, couleurs).

Rasterization

Du triangle au pixel

Le rasterizer détermine quels pixels sont couverts par chaque triangle. Il interpole les varyings (UV, normales, couleurs) à travers la surface du triangle.

Fragment Shader en Détail

Responsabilités

Le Fragment Shader calcule la couleur finale de chaque pixel :

- Sampling de textures
- Calculs d'éclairage
- Effets procéduraux

Il ne connaît rien de la géométrie — seulement des varyings interpolés.

Tests et Blending

Tests du framebuffer

- **Depth Test** : Compare la profondeur du fragment avec le Z-buffer. Si le fragment est derrière, il est ignoré.
- **Stencil Test** : Masque basé sur un buffer de stencil (ex: silhouettes, reflets).

- **Blending** : Mélange la couleur du fragment avec celle du framebuffer (transparence).